

Predicting Insurance Claim Severity Using Machine Learning

DATA6545 – Data Science and MLOps

Spring 2026

Connor Hernon

Executive Summary

This project develops a machine learning system to predict the financial severity of insurance claims using the Allstate Claims Severity dataset. The core objective is to estimate the total dollar loss associated with an insurance claim given a set of anonymized categorical and continuous claim attributes. A secondary classification model identifies claims likely to fall within the top 10% of losses, supporting downstream triage applications.

Three models were trained and evaluated: Ridge Regression as a linear baseline, Random Forest as an intermediate nonlinear benchmark, and LightGBM as the final production model. LightGBM achieved the best performance across all metrics, reaching a Mean Absolute Error of \$1,135.16, RMSE of \$1,895.83, and R^2 of 0.5595 on the held-out validation set. Five-fold cross-validation confirmed model stability with a mean MAE of \$1,145.91 and a standard deviation of just \$7.95 across folds. The secondary classifier achieved an AUC-ROC of 0.9315.

The project was carried through to full production deployment. The final system includes four serialized model artifacts, a publicly accessible Flask API deployed on Render capable of scoring individual and batch claims in real time, a Docker container for reproducibility, a version-controlled GitHub repository, MLflow experiment tracking, SHAP-based feature explainability, and an interactive Streamlit claims dashboard that visualizes the full 125,546-claim test set with severity-based highlighting. The API was verified live with end-to-end inference confirmed across all three prediction endpoints.

The most important analytical finding is that claim severity is primarily driven by a small number of categorical features, particularly cat80 and cat101, alongside the continuous variable cont14, which exhibits a highly nonlinear relationship with loss that linear models cannot capture. The system demonstrates a complete machine learning lifecycle from raw data ingestion through exploratory analysis, model selection, cross-validation, explainability, and production deployment.

1. Problem Statement and Business Context

Insurance companies face significant financial risk from claims that produce unexpectedly large losses. When high-severity claims are not identified early, insurers respond reactively, often after substantial costs have already accumulated. The downstream consequences include

inaccurate loss reserves, inefficient allocation of investigative resources, and reduced ability to intervene early on the claims most likely to produce significant financial exposure.

This project addresses the following core research question: can anonymized claim attributes be used to accurately predict insurance claim severity and identify claims likely to fall into the highest-loss segment?

The business framing has two components. The primary component is a regression system that estimates the expected dollar loss for any incoming claim. This supports reserve planning and financial forecasting by giving analysts a model-based severity estimate at the time of claim intake rather than relying solely on adjuster judgment. The secondary component is a binary classification layer that flags claims predicted to fall in the top 10% of the loss distribution for elevated review. Together these two outputs support a triage workflow where the most potentially costly claims are prioritized before significant cost development has occurred.

The dataset used is the Allstate Claims Severity dataset from Kaggle, originally published as part of a recruiting competition by Allstate Insurance Company. While the competition itself has concluded, the dataset remains publicly available and is well suited for academic study of insurance severity modeling. Importantly, this project does not follow competition rules and instead uses the data to build and evaluate a complete end-to-end machine learning pipeline oriented toward production deployment rather than leaderboard optimization.

2. Data Description

2.1 Dataset Overview

The Allstate Claims Severity dataset consists of two files. The training set contains 188,318 rows and 132 columns. The test set contains 125,546 rows and 131 columns. The only column present in training but absent from the test set is the target variable, loss, which represents the total dollar cost of the claim.

The predictor variables consist of two groups. The first group contains 116 anonymized categorical variables named cat1 through cat116. The second group contains 14 anonymized continuous variables named cont1 through cont14. All feature names are anonymized and no real-world labels are provided, which is a deliberate design choice by the competition sponsor. This anonymization presents both a practical challenge for model interpretability and a privacy advantage for academic use, since no personally identifiable information is present in the data.

A key structural property of this dataset is that it contains no missing values in either the training or test set. This simplifies preprocessing considerably and means imputation strategies, while included in the pipeline for robustness, do not materially affect the results.

2.2 Target Variable

The target variable, loss, represents the total financial loss associated with a claim in US dollars. Summary statistics for the training set are as follows:

- Mean: \$3,037
- Median: \$2,115
- Standard deviation: \$2,904
- Minimum: \$0.67
- Maximum: \$121,012
- 90th percentile: \$6,401.74

The distribution is extremely right-skewed with a skewness of approximately 3.79. The vast majority of claims fall below \$10,000, but a meaningful tail of high-severity claims extends well beyond that. This skewness is a critical modeling consideration because standard regression models assume a roughly symmetric error distribution. Training directly on the raw loss value would cause models to allocate disproportionate predictive effort to extreme observations at the expense of typical claims, where the most training signal is concentrated.

To address this, a log transformation was applied to the target variable prior to modeling. Specifically, $\log(1 + \text{loss})$ was used as the modeling target, which reduces the skewness to approximately 0.09 and produces a distribution that is close to symmetric. All model predictions were converted back to the original dollar scale using the inverse transformation $\exp(\text{prediction}) - 1$ before computing evaluation metrics. This ensures that reported metrics such as MAE and RMSE are interpretable in dollar terms rather than log-loss units.

2.3 Categorical Features

Of the 116 categorical variables, 72 have only two unique levels. Most of the remaining variables have low cardinality. However, a small number of features have substantially higher cardinality, most notably cat116 with 326 unique levels, cat110 with 131 levels, and cat109 with 84 levels. This variation in cardinality has important implications for encoding strategy. One-hot encoding of the full categorical feature set produces a very large and sparse feature matrix, which imposes significant memory and compute costs. Label encoding, by contrast, represents each category as a single integer and is well suited to gradient boosting models that handle ordinal encodings natively.

A train/test category mismatch analysis revealed that 11 categorical columns contain levels in the test set that do not appear in the training set. This is a common challenge in real-world tabular datasets and must be handled carefully to avoid errors at prediction time. In this project it is addressed by fitting encoders on the combined training and test data, ensuring all levels seen at inference time have a valid encoding. For the deployed API, which may receive claims with entirely unseen categorical values, unknown categories are handled by defaulting to index 0 as a conservative fallback.

2.4 Continuous Features

The 14 continuous variables are all scaled to similar ranges and show no meaningful distribution shift between the training and test sets. However, several pairs of continuous variables are highly correlated with one another. The strongest correlations observed are between cont11 and cont12 (0.994), cont1 and cont9 (0.930), and cont6 and cont10 (0.883). This multicollinearity is a known challenge for linear models but does not materially affect tree-based methods, since decision trees make splits on one variable at a time and are indifferent to linear dependencies across features.

Individually, the continuous variables show weak linear correlation with the target. The strongest single-variable correlation with loss is cont2 at 0.142, indicating that no single continuous feature is strongly predictive on its own. This is consistent with the expectation that claim severity is driven by combinations of features and nonlinear interactions rather than any individual attribute, and it motivates the use of ensemble methods over linear approaches.

3. Exploratory Data Analysis

3.1 Target Distribution

Plotting the raw loss distribution reveals the heavy right skew described above. The histogram shows a sharp peak near zero with a long tail extending to over \$100,000. After applying the log transformation, the distribution becomes approximately bell-shaped and symmetric, confirming that the transformation is appropriate for this modeling task. The transformation also has the practical effect of compressing the influence of extreme values during training, which improves model stability and generalization on moderate-severity claims.

3.2 Categorical Feature Analysis

Examining the cardinality of categorical variables shows that the majority are binary or near-binary. This means that for most categorical features, one-hot encoding produces only one additional column. However, the high-cardinality features such as cat116 produce a large number of sparse columns when one-hot encoded, which is computationally expensive and can negatively affect model training time. For LightGBM, label encoding was used instead, and the model was allowed to learn its own splitting criteria across the integer-encoded levels.

Grouping the training data by categorical variable values and computing mean loss per group reveals that several categorical features produce dramatically different average loss values across their levels. The features showing the strongest univariate associations with loss include cat80, cat79, cat87, cat101, and cat100. These are consistent with the SHAP results described in Section 5, which confirms that the univariate signal in these variables is preserved and amplified in the ensemble model.

3.3 Continuous Feature Correlations

A correlation heatmap of the 14 continuous variables confirms the strong pairwise relationships noted above. Clusters of highly correlated variables are visible, particularly among cont1, cont9, cont10, cont6, and cont13. For tree-based models this does not require intervention, but it would represent a significant concern for a production linear model without regularization. The presence of these clusters also makes dimensionality reduction techniques such as PCA a potentially viable preprocessing step for linear baselines, though this was not pursued in the current project.

3.4 Severity Tier Segmentation

Beyond the binary high-severity threshold, claims were segmented into four tiers based on quartile boundaries of the training loss distribution. Low-severity claims fall below \$1,000, Moderate claims fall between \$1,000 and \$3,000, High claims fall between \$3,000 and \$6,401.74, and Extreme claims fall above the 90th percentile threshold. This four-tier segmentation is used throughout the project for error analysis, dashboard visualization, and API output. It provides more operational granularity than a binary high/low split while remaining interpretable to a business audience.

3.5 Train/Test Consistency

Comparing the distributions of continuous variables between the training and test sets shows no meaningful distribution shift. Mean values and standard deviations are consistent across both sets, indicating that the train and test sets were sampled from the same underlying population. The categorical level distributions are likewise consistent, with the exception of the 11 columns noted above that contain levels in the test set not seen during training.

4. Methodology

4.1 Problem Formulation

The primary task is framed as a supervised regression problem. The target variable is $\log(1 + \text{loss})$, and predictions are converted back to the original dollar scale for evaluation. The secondary task is framed as binary classification, where a claim is labeled as high severity if its loss value falls at or above the 90th percentile of the training loss distribution, corresponding to a threshold of \$6,401.74. This threshold was computed from the training set and held fixed across all subsequent analysis, model training, and API deployment to ensure consistency between offline evaluation and live inference.

4.2 Train/Validation Split

The training data was split into a training set and a held-out validation set using an 80/20 random split with a fixed random seed of 42. This produced 150,654 training rows and 37,664 validation rows. The same split was used consistently across all models to ensure fair comparison. For the classification task, stratified splitting was used to preserve the

approximately 10/90 class ratio across both sets. The 20% held-out set was never used during model development and served as the final evaluation benchmark for all reported metrics.

4.3 Feature Encoding

The encoding strategy differed between the first and second notebook due to computational constraints encountered with one-hot encoding on the full dataset.

In the first notebook, a scikit-learn ColumnTransformer was used to apply StandardScaler and median imputation to continuous variables, and OneHotEncoder with handle_unknown set to ignore categorical variables. This approach is correct and robust but produces a very large sparse feature matrix after encoding 116 categorical columns, which contributed to the excessive cross-validation runtime experienced with the Random Forest model.

In the second notebook, label encoding was applied to all categorical variables using scikit-learn's LabelEncoder, fitted on the combined train and test data to ensure consistent mappings. This approach is appropriate for LightGBM, which handles ordinal-encoded categoricals natively and does not require one-hot encoding. Label encoding substantially reduces memory usage and training time while producing equal or better results with gradient boosting models.

A critical design decision for production deployment was to serialize all 116 fitted LabelEncoder objects alongside the model files. This dictionary of encoders, saved as label_encoders.joblib, allows the API to encode arbitrary new claims at inference time using identical categorical mappings to those used during training. Without this artifact, the API would either need to retrain encoders from scratch, requiring access to the original training data, or be unable to handle categorical features consistently. Unseen categories, which are likely in a live deployment environment, are handled by defaulting to index 0 as a safe fallback rather than raising a runtime error.

4.4 Preprocessing Pipeline

For Ridge Regression and Random Forest, preprocessing was implemented using a scikit-learn pipeline wrapping a ColumnTransformer. This design ensures that all preprocessing steps are fitted only on training data and applied consistently to validation data, preventing data leakage. The numeric transformer applies median imputation followed by standard scaling. The categorical transformer applies most-frequent imputation followed by one-hot encoding with unknown category handling. The full pipeline including the model estimator is fitted in a single call, making the entire workflow reproducible from a single script execution.

4.5 Models

Ridge Regression was used as the linear baseline. Ridge adds an L2 regularization penalty to ordinary least squares regression, which shrinks coefficients toward zero and stabilizes estimation in the presence of multicollinearity and high-dimensional feature spaces. An alpha of 1.0 was used. This model provides a principled benchmark for what a well-regularized linear

model can achieve on this dataset and establishes the performance floor that nonlinear models must exceed to justify their additional complexity.

Random Forest was used as an intermediate nonlinear benchmark. The model was trained with 150 trees, a maximum depth of 12, and a minimum sample per split of 10. Random Forest can capture nonlinear relationships and feature interactions, making it a natural step up from a linear baseline on tabular data. The model is also relatively interpretable through feature importance scores, making it a useful diagnostic tool even when it is not the final production model.

LightGBM was selected as the final production model. LightGBM is a gradient boosting framework that builds an ensemble of decision trees sequentially, with each tree correcting the errors of the previous one. It uses histogram-based splitting and leaf-wise tree growth, which makes it substantially faster and more memory-efficient than traditional gradient boosting implementations. The model was trained with 1000 estimators, a learning rate of 0.05, maximum depth of 6, 63 leaves, subsample of 0.8, and column subsampling of 0.8. Early stopping with a patience of 50 rounds was used to prevent overfitting, with the validation MAE monitored as the stopping criterion. A separate LightGBM classifier was trained for the high-severity binary task using `class_weight` set to `balanced` to account for the 10/90 class imbalance.

4.6 Cross-Validation

Five-fold cross-validation was performed on the LightGBM model using a manual fold loop rather than `cross_val_score`. This approach was chosen because it allows per-fold metric logging and avoids nested parallelism issues that caused excessive runtimes with the Random Forest. Each fold trained a fresh LightGBM model on 80% of the data and evaluated on the remaining 20%, with predictions converted back to the original loss scale before computing metrics. Cross-validation results were logged to MLflow alongside the single-split results for full experiment traceability.

5. Results and Evaluation

5.1 Regression Model Comparison

The table below summarizes performance across all three regression models on the held-out validation set.

Model	MAE	RMSE	R ²
Ridge Regression	\$1,244.68	\$2,185.06	0.4149
Random Forest	\$1,223.44	\$2,055.23	0.4823
LightGBM	\$1,135.16	\$1,895.83	0.5595

LightGBM outperforms both baseline models on all three metrics. The improvement from Ridge to LightGBM represents a reduction in MAE of approximately \$109 and an improvement in R^2 of 0.14. The progression from Ridge to Random Forest to LightGBM demonstrates the expected pattern: each step toward a more expressive nonlinear model improves performance on this dataset, confirming that claim severity is driven by nonlinear relationships and feature interactions that linear models cannot capture. The gap between Ridge and LightGBM is meaningful, at a volume of 125,546 test claims, a \$109 reduction in mean absolute error translates to millions of dollars in aggregate reserve accuracy.

5.2 Cross-Validation Results

Five-fold cross-validation on LightGBM produced the following results:

Fold	MAE	R^2	Time
1	1136.85	0.5589	19.6s
2	1157.15	0.5300	18.4s
3	1150.27	0.5611	22.1s
4	1148.42	0.5663	18.3s
5	1136.85	0.5435	19.1s
Mean	1145.91	0.5519	

Std: 7.95

The standard deviation of \$7.95 across folds is very low relative to the mean MAE of \$1,145.91, representing a coefficient of variation below 0.7%. This indicates that model performance is stable and not the result of a favorable train/validation split. Each fold completed in under 25 seconds, demonstrating the computational efficiency of LightGBM relative to the Random Forest, which required substantially longer runtimes even with a simpler model structure.

5.3 Classification Results

The high-severity classifier was evaluated at two decision thresholds: the default 0.5 cutoff and an optimized threshold derived using Youden's J statistic, which maximizes the sum of sensitivity and specificity. The AUC-ROC of 0.9315 reflects the model's overall ranking ability independent of threshold.

Results at the default 0.5 threshold:

Metric	Value
AUC-ROC	0.9315
Precision (high severity)	0.72
Recall (high severity)	0.47
F1 (high severity)	0.57
Overall Accuracy	0.83

Results at the Youden's J optimized threshold (0.0746):

Metric	Value
Precision (high severity)	0.36
Recall (high severity)	0.88
F1 (high severity)	0.51

The Youden's J threshold substantially improves recall from 0.47 to 0.88 at the cost of reduced precision, flagging a larger share of claims for review while capturing nearly nine out of ten truly high-severity claims. The appropriate threshold for a production deployment depends on the operational capacity of the claims organization, a team that can absorb more false positives in exchange for fewer missed extreme claims should prefer the Youden's J threshold. The optimized threshold and both sets of metrics were logged to MLflow for full traceability.

5.4 Feature Importance and SHAP Analysis

SHAP (SHapley Additive exPlanations) values were computed for the full validation set to identify which features drive predicted loss. SHAP provides consistent, theoretically grounded feature attributions that decompose each prediction into contributions from each input feature.

The top five most important features by mean absolute SHAP value are:

- Cat80: By far the most important feature, with a mean absolute SHAP value approximately 60% higher than the second-ranked feature
- Cat101: Strong positive association with higher predicted losses at higher encoded values
- Cont14: The most important continuous variable, showing a complex nonlinear relationship with loss

- Cat100: Moderate importance with directional effects across its levels
- Cat79: Moderate importance consistent with its strong univariate association with loss

The SHAP dependence plot for cat80 reveals that it has four encoded levels. Levels 0, 1, and 2 all push predicted loss upward, while level 3 consistently reduces predicted loss. This clean segmentation explains why cat80 is the dominant feature in the model, it partitions the claim population into a clearly lower-risk group and three higher-risk groups in a way that no other feature matches.

The SHAP dependence plot for cont14 is particularly informative. The relationship between cont14 and predicted loss is highly nonlinear, with sharp increases in SHAP value at approximately 0.25 and 0.82. This step-function pattern cannot be captured by a linear model regardless of regularization strength, and it explains in large part why Ridge Regression underperforms relative to LightGBM on this dataset. The cont14 relationship is essentially invisible to linear regression but is precisely the kind of pattern that decision tree splits are designed to capture.

5.5 Error Analysis

Examining the distribution of prediction errors on the validation set reveals several important patterns. Overall, the mean prediction error is -\$361, indicating a slight tendency to underpredict across all claims. The median error is -\$27, suggesting that the distribution is skewed by a small number of large underpredictions rather than a systematic negative bias across the full population.

Breaking down performance by severity tier exposes the most important limitation of the model:

Tier	Claims	Mean Actual	Mean Predicted	Mean Abs Error	Mean % Error
Low (< \$1,000)	6,726	\$722	\$1,339	\$628	103.7%
Moderate (\$1,000–\$3,000)	17,830	\$1,837	\$1,987	\$633	36.0%
High (\$3,000–\$6,401)	9,339	\$4,328	\$3,443	\$1,436	33.1%
Extreme (> \$6,401)	3,769	\$9,558	\$6,321	\$3,672	37.2%

The model performs best on Moderate claims in absolute dollar terms, which is expected since this tier represents the largest training segment and is closest to the mean of the log-transformed distribution. For Low claims, the model overpredicts substantially in percentage terms, predicting roughly twice the actual loss on average, though the absolute dollar amounts involved are small. For Extreme claims, the model underpredicts by an average of \$3,237, which is the most consequential error pattern for a real insurance application where reserve adequacy on large claims is critical.

This regression-to-the-mean behavior is expected and well documented in severity modeling literature. The model is pulled toward the center of the training distribution and struggles with the most extreme observations where training data is sparse. A quantile regression approach or a two-stage model that separately handles extreme claims could partially address this limitation.

5.6 Test Set Predictions

Predictions were generated for all 125,546 test claims. The severity tier distribution of test predictions is as follows:

Tier	Count	Percentage
Low	9,784	7.8%
Moderate	80,756	64.3%
High	28,049	22.4%
Extreme	6,957	5.5%

This distribution is consistent with the training data distribution, indicating that the model generalizes without significant distributional shift to the test set. The proportion of Extreme predictions at 5.5% is slightly below the 10% threshold used to define the high-severity class, reflecting the model's documented tendency to underpredict at the extremes of the loss distribution.

6. Technical Artifacts and Deployment

6.1 Model Artifacts and Serialization

Four serialized artifacts were produced and version-controlled as part of this project. The LightGBM regressor is saved as `lgbm_regressor.joblib` and the LightGBM classifier is saved as `lgbm_classifier.joblib`. These are the core inference models and are loaded once at API startup.

A third artifact, `label_encoders.joblib`, stores a Python dictionary mapping each of the 116 categorical column names to its fitted `LabelEncoder` object. Saving the encoders as a separate artifact was a deliberate production-oriented design decision. Without this file, the API would be

unable to correctly encode incoming claims at inference time, since the integer mapping for each categorical level is only valid relative to the encoder fitted during training. For example, the value "D" in cat80 maps to a specific integer that must match what the model saw during training; any deviation produces incorrect predictions. The encoder artifact ensures this mapping is preserved exactly across the training-to-deployment boundary.

A fourth artifact, `metadata.joblib`, stores the complete feature column list in training order, the categorical and continuous column sublists, the high-severity threshold dollar value, the tier boundary values, and the validation metrics. This metadata file allows the API to validate incoming requests, assign severity tiers to predictions, and expose model performance information through the `/model_info` endpoint without requiring hardcoded values in the application code.

6.2 Flask API

The trained models are served through a Flask application with four endpoints. The `/health` endpoint returns a liveness confirmation and the model name, allowing orchestration systems to verify that the service is running. The `/model_info` endpoint returns model metadata including the number of features, severity tier boundaries, high-severity threshold, and validation performance metrics. This endpoint is useful for downstream consumers who need to understand the model's behavior and expected performance without reading the project documentation.

The `/predict` endpoint accepts a single claim as a JSON object with any combination of the 130 feature fields and returns a structured prediction response. Missing features default to safe values, zero for continuous features and the most common encoded category for categorical features, allowing clients to submit partial claims without causing errors. The endpoint also validates continuous feature values and returns a warnings list when values fall outside the expected range.

An example response from the `/predict` endpoint is shown below:

```
json
{
  "predicted_loss": 2268.11,
  "severity_tier": "Moderate",
  "high_severity_flag": false,
  "high_severity_probability": 0.0371,
  "warnings": []
}
```

The `/predict_batch` endpoint accepts a JSON object containing a list of up to 500 claims and returns predictions for all of them in a single response. Each result in the batch response includes an index field identifying its position in the input list, enabling clients to match

predictions back to their original claims. Batch processing is substantially more efficient than sequential single-claim requests for high-volume scoring use cases.

The API was verified live using a full 130-feature claim payload constructed with default values. The `/health` endpoint returned a 200 status with the correct model name. The `/predict` endpoint returned a predicted loss of \$2,268.11 with severity tier Moderate and a high-severity probability of 3.71%. The `/predict_batch` endpoint successfully processed a two-claim batch and returned independent predictions for each. All responses were received with latency consistent with a model loaded in memory, confirming that the artifact loading at startup was successful.

6.3 Docker Container

The application is containerized using Docker to ensure reproducibility across environments. The Dockerfile uses a Python 3.11-slim base image, installs `libgomp1` as a system dependency required by LightGBM, copies the pinned requirements file and installs all Python dependencies, then copies the application code and model artifacts. The container exposes port 8080 and is started using Gunicorn with two worker processes and a 120-second timeout to accommodate model loading time.

Containerization means that any environment with Docker installed can run the identical inference service without dependency conflicts, Python version mismatches, or missing system libraries. The Docker image was built and verified prior to Render deployment to confirm that the containerized application behaved identically to the local development version.

6.4 Render Deployment

The API is publicly deployed on Render at `allstate-severity-api.onrender.com`. Render was configured using a `render.yaml` file committed to the repository, which specifies the build command, start command, environment variables, and health check path. The health check is configured to poll `/health` every 30 seconds, allowing Render to automatically restart the service if it becomes unresponsive.

Automatic deployment is enabled, meaning every push to the main branch of the GitHub repository triggers a new build and deployment. This creates a continuous deployment pipeline where code changes are reflected in the live API without manual intervention. The initial deployment required debugging a path error in the Flask application, the model artifacts were originally referenced using absolute Colab file paths that did not exist in the container environment, and were corrected to use relative paths pointing to the `models/` directory bundled with the Docker image.

6.5 GitHub Repository

All project code, model artifacts, API source, and deployment configuration are version-controlled at `github.com/hernon33/allstate-severity-api`. The repository is organized with model artifacts in a `models/` subdirectory, the Flask application in `app.py` at the repository root, and deployment configuration files including `Dockerfile`, `render.yaml`, `requirements.txt`, and

.gitignore also at the root. A comprehensive README documents the project background, model performance, API endpoints with example requests and responses, reproduction instructions, and limitations.

The model artifact files are committed directly to the repository. LightGBM serialized models are compact relative to other ensemble methods, making direct Git storage practical without requiring Git LFS or external artifact storage. The repository is public, allowing the instructor and any other reviewer to clone the project, install dependencies from the pinned requirements file, and run the API locally using a single command.

6.6 Interactive Claims Dashboard

An interactive claims dashboard was developed as a companion Streamlit application to support visual exploration of the full test set predictions. The dashboard is publicly deployed at allstate-claims-dashboard.streamlit.app and maintained in a separate repository at github.com/hernon33/allstate-claims-dashboard.

The dashboard loads all 125,546 test predictions from a CSV committed to the repository, which contains the claim ID, predicted dollar loss, high-severity classifier probability, high-severity flag, and severity tier for every test claim. All predictions were generated by running both the LightGBM regressor and classifier against the encoded test set using the same serialized artifacts deployed in the API, ensuring complete consistency between API inference and dashboard results.

The dashboard presents five summary metric cards showing total claims, average predicted loss, percentage of claims flagged as high severity, total extreme claim count, and the number of claims currently matching the active filters. Three charts provide distributional context: a bar chart of claims by severity tier, a histogram of predicted loss across eight dollar ranges, and a full histogram of classifier probabilities across all 125,546 claims annotated with the AUC-ROC and Youden's J threshold.

The main body is a paginated claims table displaying 25 records per page with dollar-formatted predicted loss, a scaled progress bar showing high-severity probability as a percentage, a flagged indicator, and severity tier. The sidebar supports filtering by severity tier, flag status, and loss range, as well as direct claim ID search. The dashboard is built entirely with Streamlit and Plotly with no external dependencies beyond those pre-installed on Streamlit Community Cloud.

6.7 MLflow Experiment Tracking

All four model runs, Ridge Regression, Random Forest, LightGBM regressor, and LightGBM classifier, were logged to MLflow with full parameter and metric tracking. Logged parameters include model type, hyperparameters, target transformation, encoding strategy, and dataset dimensions. Logged metrics include validation MAE, RMSE, R^2 , cross-validation mean and standard deviation, classification threshold method, and high-severity error statistics. A run summary is committed to the repository as `mlflow_run_summary.csv`, providing a permanent

record of all experiment results without requiring the full MLflow tracking directory to be version-controlled.

7. Limitations and Future Work

7.1 Limitations

Anonymized features. All 130 predictor variables are fully anonymized. While the model achieves reasonable predictive accuracy, it is not possible to provide meaningful business interpretation of the features. A claims professional cannot act on the finding that cat80 level 3 reduces predicted loss without knowing what cat80 actually represents in terms of claim characteristics. This limits the operational utility of the SHAP explanations beyond academic demonstration. In a real insurance application, feature transparency would be a regulatory and practical requirement.

Underprediction on extreme claims. The model systematically underpredicts the most severe claims. For an insurer, extreme claims are often the most financially consequential and the ones where accurate prediction matters most. A mean underprediction of \$3,237 on claims above \$6,401 represents a meaningful gap for reserve planning purposes. A point estimate model of this type structurally cannot fully capture tail risk, which motivates the quantile regression approach described in future work.

Recall on high-severity claims at the default threshold. At the default 0.5 decision threshold the classifier captures approximately 47% of truly high-severity claims. While threshold optimization via Youden's J improves recall to 0.88, this comes at the cost of reduced precision and a higher volume of claims flagged for review. The appropriate tradeoff must be calibrated to the operational context of the claims organization.

Cross-sectional data. The dataset is cross-sectional and contains no time-based features. It is therefore not possible to model claim development over time, reserve changes, or lifecycle dynamics that are central to real insurance operations. A production severity model would ideally incorporate claim age, development factors, and historical loss trends.

No external validation. Because the dataset is fully anonymized and the ground truth for the test set is not publicly available from Kaggle, it is not possible to validate predictions on a truly independent dataset beyond the held-out training split. The cross-validation results provide confidence in model stability, but they cannot substitute for validation against real-world claims data.

No drift monitoring. The deployed API does not currently include input distribution monitoring. In a production environment where claim characteristics may shift over time due to changes in underwriting appetite, geographic exposure, or claim handling procedures, model predictions can degrade silently. A drift monitoring layer that tracks the distributions of incoming feature

values relative to the training distribution would be a necessary component of a production-grade deployment.

7.2 Future Work

Several directions would strengthen this work given additional time and resources. First, hyperparameter tuning using a systematic search such as Optuna or Bayesian optimization could improve LightGBM performance beyond what was achieved with the manually selected parameters used here. The current hyperparameters were chosen based on domain convention and manual experimentation; a grid search over learning rate, number of leaves, and regularization parameters would likely yield additional gains.

Second, a quantile regression approach could be used to generate prediction intervals rather than point estimates. LightGBM natively supports quantile regression through its objective parameter. Producing a 10th and 90th percentile prediction alongside the point estimate would give the model's output a probabilistic interpretation that is directly useful for reserving, where an insurer typically needs to hold reserves at a confidence level above the expected value.

Third, stacking the regression and classification models, using the regression output as an additional feature in the classifier, could improve recall on high-severity claims. The regression model's predicted loss is a direct indicator of severity and carries information that the classifier currently must derive independently from the raw features. A second-stage classifier trained on both raw features and the regression prediction may achieve better recall without sacrificing precision.

Fourth, retraining the model on the full training set rather than an 80% split before generating final test predictions would make full use of the available data. The current approach withholds 37,664 rows for validation, which represents a meaningful portion of the training signal.

Fifth, implementing a feature drift monitoring component in the API would improve production reliability. An approach based on the Population Stability Index computed across incoming feature distributions relative to the training baseline would provide early warning of data quality issues or population shifts that could degrade model performance.

8. Ethical Considerations

8.1 Potential Harms

A claim severity prediction system, if deployed in a real insurance context, could influence how claims are handled, reviewed, and resourced. If the model systematically underpredicts severity for certain groups of claimants, those individuals may receive less attention or fewer resources than their claims warrant. The regression-to-the-mean behavior documented in this project, where extreme claims are underpredicted by an average of \$3,237, represents one such pattern that could disproportionately affect claimants with legitimate high-cost claims. A triage system

that relies on model predictions to allocate adjuster attention could therefore systematically underserve the claimants with the highest actual need.

8.2 Bias Assessment

Because all features are anonymized it is not possible to directly assess whether the model discriminates against protected groups such as those defined by race, age, gender, or geography. Categorical features such as cat80, which is the single most predictive variable in the model, could in principle encode geographic region, vehicle type, claim type, or any number of attributes that may correlate with protected characteristics. In a real deployment this assessment would be a critical regulatory requirement. Any production use of a model like this would require de-anonymizing features and conducting a full disparate impact analysis before deployment, and the results would need to satisfy applicable fair lending and fair claims handling regulations.

8.3 Privacy

The Allstate dataset used in this project is a public Kaggle dataset and contains no personally identifiable information. All fields are anonymized by the original data provider. No privacy concerns arise from the academic use of this dataset. The deployed API does not log or store any claim data submitted by users, and the public GitHub repository contains no sensitive information beyond the model artifacts themselves.

8.4 Misuse Potential

A severity prediction model with an AUC of 0.93 is sufficiently powerful to meaningfully alter claim handling workflows. It could potentially be misused to preemptively deny or delay claims based on predicted cost rather than actual merit, to selectively assign lower-quality service resources to claims predicted to be low severity, or to identify claimants predicted to have high-cost claims for targeted investigation. Any deployment in a regulated environment should include human oversight at the decision layer, documented appeal mechanisms for claimants whose claims are affected by model-driven decisions, and regular audits of model-driven outcomes against fairness benchmarks. The model should be positioned as a prioritization support tool rather than a decision-making system.

8.5 Transparency

Given that all features are anonymized, individual predictions cannot be explained to affected claimants in meaningful terms. The SHAP values produced by this model identify which encoded features drove a given prediction, but without knowing what those features represent in the real world, explanations are not actionable for claimants, adjusters, or regulators. In a regulated insurance environment, explainability requirements may make a model of this type difficult or impossible to deploy without de-anonymization and additional interpretability work. The API's /predict endpoint includes a warnings field that surfaces data quality concerns in real time, but full explanation capability would require feature de-anonymization outside the scope of this project.

9. Conclusion

This project successfully developed, evaluated, and deployed a machine learning system for predicting insurance claim severity using the Allstate Claims Severity dataset. LightGBM was identified as the strongest model, achieving a validation MAE of \$1,135, RMSE of \$1,896, and R^2 of 0.56, with stable performance confirmed across five cross-validation folds with a standard deviation of just \$7.95. A secondary classifier for identifying high-severity claims achieved an AUC of 0.93, providing a strong ranking signal for triage prioritization. Threshold optimization using Youden's J statistic was applied to the classifier, improving recall from 0.47 to 0.88 at the cost of reduced precision, a tradeoff documented and logged for operational calibration.

The analysis produced several meaningful findings about the structure of claim severity. Severity is primarily driven by a small number of categorical features, with cat80 being the single most predictive variable by a significant margin. The continuous variable cont14 exhibits a highly nonlinear relationship with loss that is invisible to linear regression but precisely captured by gradient boosting splits, explaining the substantial performance gap between Ridge Regression and LightGBM. The model performs well on typical moderate-severity claims but systematically underpredicts extreme losses, which represents the most important practical limitation for reserve planning applications.

Beyond modeling, the project delivered a complete production system. Four serialized model artifacts are stored in a public GitHub repository alongside a containerized Flask API deployed on Render at allstate-severity-api.onrender.com, where all four endpoints have been verified with live inference. An interactive Streamlit dashboard provides visual exploration of the full 125,546-claim test set with severity-based highlighting and filtering. MLflow experiment tracking provides full reproducibility of all four model runs, with a run summary committed directly to the repository. Together these components demonstrate a complete machine learning lifecycle, from raw data ingestion through exploratory analysis, model selection, cross-validation, explainability, containerization, and live cloud deployment, carried through to a working, accessible production system.

References

Allstate Insurance Company. (2016). Allstate Claims Severity. Kaggle.
<https://www.kaggle.com/c/allstate-claims-severity>

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30.

Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesneau, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.